

개발 가이드

‘22.06.09 

1. Overview	3
2. 설계	3
3. 구현	3
3.1. 공통 코딩 표준	3
3.2. IDE	5
3.2.1. 종류	5
3.2.2. 설정	5
3.3. Back-End	5
3.3.1. 환경	5
3.3.2. 패키지 구조 & TODO Java 네이밍 규칙	5
3.3.3. MyBatis	7
3.3.4. RESTful API	8
3.3.5. 용도별 Java Library	8
3.3.6. 테스트	8
3.3.7. Code Quality 검사	9
3.3.8. MySQL	9
3.3.9. 범용 로직	10
3.4. Front-End	11
3.4.1. 코딩표준	11
3.4.2. jQuery	11
3.4.2. 공통ajax모듈	11
3.4.3. RESTful API에 post로 JSON객체와 파일첨부를 동시에 전송할 때 샘플	12
3.5. Database	13
3.5.1. Database 설정	13
3.5.2. DB관리 추천 툴	14
3.6. 인증(Authentication)과 인가(Authorization)	14
3.7. 반영	14
4. 산출물 관리	14
4.1. 기획	15
4.2. 설계	15
4.3. 개발	15
5. 유지/보수	15
5.1. 태스크월드	15
5.2. 개발 서버	16
6. 기타	16

1. Overview

해당 문서는 사내 개발 프로젝트를 진행하기 위한 전반적인 규칙과 추천 기술들을 기술하여 모든 개발자가 정형화된 규칙을 지침으로써, 프로젝트 및 소스 분석에 허들을 없애고 유지 보수가 원활하게 진행되도록 하는데 목적이 있다.

2. 설계

1. URL 네이밍 : 설계 이전에 2차 도메인까지 확정되어 있을 것이므로 3차 도메인 이후부터 네이밍

a. 도메인 : 좁은 범위에서 넓은 범위 순서로 작성

ex)

`https://del-api[redacted]co.kr`

`https://sett[redacted]co.kr/api/~`

`https://api[redacted]co.kr/sett/v1.0`

`https://api[redacted]co.kr/del/v1.0`

#	용도	주소
1	[redacted] api	<code>https://sett-api[redacted]co.kr</code>
2	[redacted] pc웹	<code>https://sett-owner[redacted]co.kr</code> (편의를 위해 www제거할 수 있음)
3	[redacted] 앱 웹뷰	<code>https://sett-app[redacted]co.kr</code>
4	[redacted] 웹	<code>https://sett-admin[redacted]co.kr</code>

b. path(도메인 명 오른쪽의 경로) : 넓은 범위에서 좁은 범위(상하/선후/소유관계) 순서로 정하되 시스템별로 설계 단계에서 사전 정의 할 것, 대소문자 구분

ex)

i. [redacted] api > [redacted] > [redacted] 상세 조회 :

`https://api.sett\[redacted\]co.kr/v1.0/owners/\[redacted\]d/specs/\[redacted\]`

`https://api.sett\[redacted\]co.kr/v1.0/specs`

ii. [redacted] pc웹 > 고객지원 > FAQ 화면 :

`https://www.sett\[redacted\]co.kr/support/faq`

2. 버전 관리 *TODO* 필요여부검토

a. 관리 규칙 : *TODO* 명시화 필요

b. 버전번호 관리 : semantic versioning 방식을 따름 *TODO*구체적인 가이드 추가

3. 구현

3.1. 공통 코딩 표준

일반적인 사항은 자바 코딩 규칙을 따름. [『프로그램 표준 코딩 규칙』행정안전부](#)

1. 지양해야 할 사항

- 한 글자 변수명은 가독성이 떨어지고 틀에서 검색하거나 클릭하기 힘들 ex) i,j,k
- 용도를 설명하기에 부족한 일반적인 단어
ex) info, data, value, input, output
- 클래스(테이블)명과 동일한 인스턴스명
ex) carTable 보다는 car
ex) car 보다는 brokenCar : 형용사를 붙여서 의미를 명확하게 표현
- 배열형은 List를 붙인다, 그 외의 모든 변수에는 Data를 붙인다
- 마법의 숫자를 사용하지 않는다. 상수는 변수를 선언해서 사용. 참고 : <https://javabom.tistory.com/28>

2. 많이 쓰이는 변수명

- 구분자 : ~type(한글을 영어발음으로 사용금지 ex. gubun)
- 순번/번호 : num
- 개수 : cnt
- 크기 : size
- 상태 : status
(어떤 상태코드로 정의 가능. ex)exit status, http status)
- 통계 : stats(statistics의 약어)
- 현황 : state
- 시점에 따른 구분
 - org : 이전의, 원래의(old를 쓰지 않음)
 - new : 새로 생성되는 경우 예전 변수와 구분이 필요할 때

3. boolean 변수명 접두어

- is : ~인지 판별
 - is + 명사 : isParent
 - is + 현재진행(~ing) : isExecuting, isEditing
 - is + 형용사/과거분사 : IsNew, isHidden, isSelected, isCompleted
 - is + 동사원형 : 의미가 명확하지 않으므로 사용하지 않는다
 - isEdit(X) 🖱️ isEditable(O) : 편집할 수 있는가
 - isEditing : 편집 중 인가
 - canEdit : 편집할 수 있는가
- 조동사 ex) can : '~할 수 있는가' ex) should : '~해야 하는가'
 - canCommit : 커밋 가능한가
 - shouldReset : 리셋해야 하는가
- has : ~을 가지고 있는가
 - hasiCloudAccount : iCloud 계정을 가지고 있는가
 - hasVideo : 동영상 포함하고 있는가
- 동사 : 3인칭 단수형 사용
 - supports : ~을 지원하는가? (ex. supportsLogin : 로그인을 지원하는가)
 - includes : ~을 포함하는가? (ex : includesTax : 세금을 포함하는가)
 - shows : ~을 보여줄 것인가? (ex. showsId : id를 보여주는가)
 - allows : ~을 허용할 것인가? (ex. allowsTransfer : 이전 가능한가)

- v. accepts: ~을 받아 주는가? (ex. acceptsCash : 현금을 받는가)
 - vi. contains: ~을 포함하고 있는가? (ex. containsAddress : 주소가 포함되어 있는가)
4. 들여쓰기 준수 : 탭문자 사용, 스페이스 사용 금지(YAML의 경우만 스페이스 문자 2개 사용)
 5. 저장할 때 문장 끝의 모든 공백문자 제거(대부분의 툴에서 지원함)

3.2. IDE

3.2.1. 종류

1. STS(Spring Tool Suite) :
 - a. 버전 : 4
 - b. Download : [link](#)
 - c. Spring framework 관련 개발 툴
2. Visual Studio Code
 - a. [코드 가독성 높이기](#)
 - b. 필수 extension
 - i. [Extension Pack for Java](#) : VS Code에서 자바 개발을 위한 필수 익스텐션 모음
 - ii. [Lombok Annotations Support for VS Code](#) : lombok어노테이션 사용지원
 - c. 추천 extension
 - i. [SonarLint](#) : 코드 퀄리티 향상
 - ii. [Git Branch Warnings](#) : main 브랜치에서 개발 시 경고 보여주기
 - iii. [GitLens — Git supercharged](#)
 - iv. [REST Client](#) : REST api 테스트용
 - v. [Native-ASCII Converter](#) : properties파일의 한글 유니코드 출력
 - vi. [Live Server](#) : 개발용 웹서버, 파일 저장 시 자동 리로딩 지원, 순수 html + javascript 기반 웹개발 할 때 사용

3.2.2. 설정

1. Tab size: space 2칸으로 설정
2. Fonts : D2Coding ([Download](#))
3. 주석 format 결정

파일 단위 주석

```
/**
 *
 *
 * @Project : ${project_name}
 * @FileName : ${file_name}
 * @Date   : ${date}
 * @Author : ${user}
 *
 * ${tags}
 *
 * <pre>
 * [HISTORY]
 * Date      Author   Description
 * -----
```

<pre> * * * </pre> */ </pre>
Method 단위 주석
<pre> /** * * * @Method : \${enclosing_method} * @Date : \${date} * @Author : \${user} * * \${tags} * * <pre> * [HISTORY] * Date Author Description * ----- * * * </pre> */ </pre>

3.3. Back-End

3.3.1. 환경

1. Java Version : 11
2. Spring Boot 2.6.8('22.06.23현재) : [Spring Initializr로 프로젝트 생성하기](#)
3. 의존성관리 Build툴 : gradle-7.4.1(버전확인 방법 : ~/gradle/wrapper/gradle-wrapper.properties 의 distributionUrl 확인)

3.3.2. 패키지 구조 & TODO Java 네이밍 규칙

Package tree			Description
com[블랙박스] {프로젝트 약어}			프로젝트 기본 package
	conf		설정 정보 관련 package
	controller		controller 관련 package
	domain		도메인 모델 관련 package
	service		비즈니스 로직 관련 package

	mapper		DB 쿼리 mapping 관련 package
	comm		공통 로직 관련 package

1. 최상위 패키지 : com[redacted]{해당프로젝트명 약어}
2. ~/ : 메인 java만 있음({해당프로젝트명}Application.java)
3. ~/conf :
 - a. 네이밍 규칙 : {설정명}conf.java
 - b. 각종 Spring 설정 java(인증, 인가 등. @Configuration 어노테이션을 사용하는 모든 클래스)
4. ~/controller
 - a. 네이밍 규칙 : {업무명}controller.java
 - b. 입력 인수 : Request객체
유효성 확인. 그 외 비즈니스 로직은 넣지 않음
 - c. 출력 타입 : Response객체
 - i. Service혹은 Mapper를 호출 한 후 반환 값을 Response 객체로 변환 후 반환
 - ii. 오류 시 메시지 생성
5. ~/domain
 - a. model : 도메인 모델.
 - i. 모듈간의 입출력 인수로 사용.
 - ii. 해당 도메인 모델의 고유 로직 구현
로직 중복을 방지하고 로직을 모아둘 수 있어 재사용성 향상.
단 Lombok의 setter, getter를 사용할 경우 도메인 로직 적용이 안될 수 있으므로 주의 필요
[도메인 모델 로직 샘플]
 1. 식별자 생성 로직(ex. 앨범 코드 생성, 트랙 코드 생성...)
 2. 상태 변경 유효성 검증 로직(ex. 접수상태에서 서비스 중지 상태로 변경 불가)
 3. 필드별 값 유효성 체크 로직(ex. 트랙 객체 생성 시 앨범 객체는 필수, 판매 금액은 양수...)
 4. 데이터 생성 로직(ex. 외부 전송용 앨범명, 트랙명 생성 로직)
 - b. dto : 모듈간 입출력에 도메인 모델을 인수로 사용하기 부적절한 경우
6. ~/service
 - a. 네이밍 규칙 : {업무명}service.java
 - b. interface를 따로 만들지 않고 단일 클래스로 개발
 - c. 입력 인수
 - i. 단건 CRUD : 도메인 모델
 - ii. 목록 조회 : DTO
 - d. 출력 타입
 - i. 단건 조회 : 도메인 모델
 - ii. 목록 조회 : DTO
 - iii. 그 외(처리 성공 여부, 특정 필드만 조회 등...) : DTO, Java객체(HashMap, String...), 원시 타입
 - e. 비즈니스 로직 구현(but 특정 도메인 모델 고유의 로직은 해당 도메인에 구현)
 - f. Mapper와 다른 Service를 호출 할 수 있음
 - g. 트랜잭션 관리(@Transactional 클래스 어노테이션 추가)
7. ~/mapper(Mybatis)
 - a. 네이밍 규칙 : {업무명}mapper.java
 - b. DB 쿼리 수행

- c. 입력 인수
 - i. 단건 CRUD : 도메인 모델
 - ii. 목록 조회 : DTO
 - d. 출력 타입
 - i. 쿼리 결과에 비즈니스 로직을 적용해야 할 때 : 도메인 모델
 - ii. 쿼리 결과를 바로 반환할 때 : 도메인 모델, DTO, Java객체(HashMap, String...), 원시 타입
8. ~/comm
- a. Money.java : 각종 금액 표현용 객체
 - i. setAmount(BigDecimal amount) : null, 음수 체크 로직 있음

3.3.3. MyBatis

1. 상하관계가 복잡한 클래스의 데이터 한 건 select시 [advanced-result-maps](#) 사용
2. 단건 조회시 Optional을 사용하면 null 검사를 깔끔해짐 🙌 [참고](#)

```
Optional<Order> findOne(Integer id);
Order order = orderOpt.orElseThrow(() -> new NoOrderException());
```

3. sql xml
 - a. 주석사용 : 쿼리별로 쿼리아이디를 SQL 쿼리 주석으로 넣는다. 컬럼 주석은 넣지 않는다.

```
<insert id="insertArtistKeywordInfo"
parameterType="com.███.admin.dto.ArtistKwdInqRsltDto">
  /* insertArtistKeywordInfo */ 🙌 필수 포함
  INSERT INTO ██████████ KEYWORD ( /* T 아티스트 키워드 정보 관리 */ 🙌 이
주석 제거
    ACCOUNT_ID /* 사용자 내부 관리 번호 */ 🙌 이 주석 제거
    , ARTIST_ID /* 아티스트 unique ID */ 🙌 이 주석 제거
```

- b. 비어있는 값 체크방식
[추천] Apache StringUtils의 isEmpty사용

```
<if test="@org.apache.commons.lang3.StringUtils@isEmpty(ctgId)">
  AND A.CATEGORY_ID = #{ctgId}
</if>
```

[비추천]

```
WHERE A.CODE_VALUE1 = #{grpId}
<if test='ctgId != null or ctgId != ""'> → 사용하지 않음
  AND A.CATEGORY_ID = #{ctgId}
</if>
```

- c. enum비교 방법(equals사용하지 말것, 동작하지 않음. toString하지 않고 비교 시 동작할 때도 있고 안할 때도 있음)

```
<when test="status.toString() ==
@com.███.sett.file.domain.seller.raw.RawSalesDataStatus@InProgress.toStri
ng()">
```


- d. 코드값을 하드코딩 할 때

```
status =  
'${@com.████.sett.file.domain.seller.raw.RawSalesDataStatus@Error.getCode()}'
```

e.

4. Mybatis Generator :

- a. 테이블 정보에서 Mybatis 소스 자동 생성(참고:<https://mybatis.org/generator>)
- b. Download : [link](#)

3.3.4. RESTful API

- 1. Client 개발팀과의 소통은 swagger를 통한 API 문서(web based)를 기반으로 한다.
단, swagger는 개발 서버에서만 활성화 한다. (상용서버에서는 노출되지 않도록 한다.)
- 2. 참고 문서 : [api설계_구현 가이드.docx](#)

3.3.5. 용도별 Java Library

- 1. 공통 후처리(post-process)
각종 비즈니스 로직에 공통된 후처리(메시지 전송, 파일 생성, 외부 연동...)을 메인 로직과 분리하여 처리하기 위한 방법 **TODO**샘플추가 후 더 논의
 - a. ~~AOP(Aspect Oriented Programming)~~
 - b. Queue : 비동기 처리를 위해
- 2. 스케줄링 : org.springframework.scheduling
- 3. 멀티쓰레드 : org.springframework.core.task.TaskExecutor
- 4. 각종 문자열 처리 : Apache StringUtils
- 5. 날짜/시간 : java.time사용(java.util.Date, Calendar는 지양)
- 6. 금액 계산 : java.math.BigDecimal
- 7. 객체 속성 복사 : org.springframework.beans.BeanUtils
객체 속성들을 복사할 때 사용(클래스가 다르더라도 필드명이 같으면 쉽게 복사가능)
- 8. getter, setter : Lombok사용
- 9. 로깅 : slf4j(System.out, Exception.printStackTrace() 지양) **TODO log.error** 확인필요
 - a. 설정 파일 : application.yml(logback-spring.xml사용금지)
 - b. 프로파일에 따라 로깅을 다르게 처리하기
<https://gitlab.com/oz-z/ozz-dsp-sender/-/blob/main/src/main/resources/application-local.yml>
- 10. JSON : **TODO**추가 각자 리서치하여 선정('22.06.17)
- 11. Template Engines
텍스트 형태의 템플릿을 사용하여 데이터를 출력하는 엔진 라이브러리를 의미한다.
뷰 템플릿 엔진으로 많이 사용중이며, 텍스트 기반의 템플릿에 데이터를 추가하는 엔진으로도 사용한다.
 - a. FreeMarker
 - [프리마커\(FreeMarker\)](#) 템플릿 엔진은 스프링 부트에서 공식적으로 지원하는 템플릿 엔진 중 하나입니다.
 - 텍스트 기반의 템플릿에 사용하기를 추천한다.
 - 프리마커 템플릿 엔진을 공식적으로 지원하는 스프링 프레임워크에는 SpringTemplateLoader라는 구현체를 포함하고 있습니다. ([Refer](#))
 - [SpringTemplateLoader](#)
 - [StringTemplateLoader](#)

- [MultiTemplateLoader](#)

b. Thymeleaf

i. 뷰 템플릿에 많이 사용된다.

12. CSV

a. [lightning csv](#) : 속도가 빠르고 사용하기 편함

13. 추가로 필요한 공통 라이브러리는 개발 회의를 통해 선정

3.3.6. 테스트

최대한 테스트를 상세하게 많이 작성함

1. unit test

a) 라이브러리 : Junit 🖱️ [리포트 샘플](#)

b) mocking 라이브러리 : Mockito

c) 테스트 메소드에 @DisplayName 추가

```
@Test
@DisplayName("년도가 [REDACTED] 과 같거나 큰 양수 일 때 정상동작
테스트")
public void setValidYearTest() {
    ...
}
```

2. test coverage report tool : 아래 둘 중 하나 사용

a) [Java Code Coverage for Eclipse](#)

b) [Jacoco](#) : [build.gradle 설정 샘플](#), [리포트 샘플](#)

3. API 자동화 통합 테스트 : [Karate](#)

여러 api를 순차적으로 호출할 때 원하는 결과가 나오는지 테스트 자동화

[구동방법]

a) Junit test suite : Maven, Gradle 을 이용하여 의존성을 추가한 후 junit 테스트를 작성하여 task형태로 실행하는 방법. 혹은 Eclipse, IntelliJ 같은 IDE의 플러그인을 이용해서 실행

b) stand-alone : jar로 터미널에서 실행하는 방법

c) [참고 : Karate, RESTful API 시나리오기반 자동화 테스트 데모](#)

3.3.7. Code Quality 검사

TODO 선정필요(PMD? SonarQube? SonarLint?)

4. SonarLint

a) eclipse용 : <https://www.sonarlint.org/eclipse/>

b) vs code용 : <https://www.sonarlint.org/vscode>

3.3.8. MySQL

Database 중 MySQL 사용을 위한 일반적인 방법 및 사용 툴에 대한 내용을 기술한다.

1. DB 접속 추천 툴

a. DBeaver :

가격	무료
----	----

다운로드	Link
특징	cross platform, jdbc드라이버가 있는 모든 DB사용가능 but 특정 DB에 특화된 기능은 부족함

b. SQLYog :

가격	무료
다운로드	Link
특징	

2. 개발관련

a. DB pool설정

- i. *TODO*추가 필요 *Hikari* 설정방법, 샘플 추가

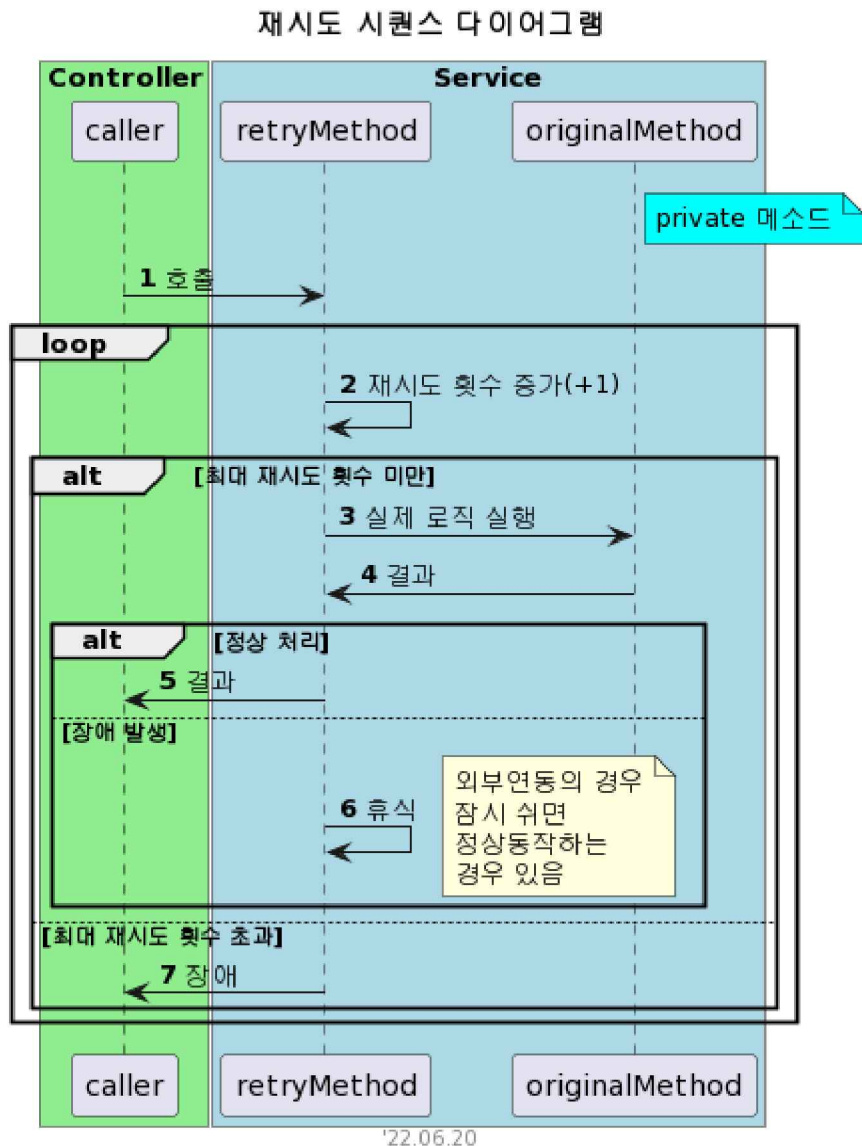
3.3.9. 범용 로직

1. 재시도

어떤 이유로 일시적 장애가 발생하여 로직을 처리 못하는 경우 일정 횟수만큼 재시도해야 할 때 사용.

[spring-retry](#)를 사용하면 깔끔하기는 하지만 *spring*에 의존해야 하고 로직이 복잡해지므로 다음

방식을 고려해 본다.



2.

3.4. Front-End

3.4.1. Javascript 코딩 표준

1. 기본 코딩 표준은 위의 항목 [3.1. 공통 코딩 표준](#)을 준수할 것
2. 변수
 - a. 변수 선언시 `const`와 `let`을 사용
 - i. 변수 중복 선언을 방지
 - ii. `var`로 선언된 변수를 전역 변수로 사용시 전역 객체(`window`)의 프로퍼티가 됨

```
var foo = 123; // 전역변수
console.log(window.foo); // 123
```

```
let foo = 123; // 전역변수
console.log(window.foo); // undefined
```

- b. 선언시 `const`를 그룹화 하고 다음 `let`을 그룹화 함
- c. `var` 사용은 지양함

3. 오브젝트

- a. 오브젝트를 작성시 리터럴 구문을 사용
 - i. `Object` 생성자 함수로 객체를 생성하는 것을 단순화시킨 축약 표현
 - ii. 객체 리터럴 구문 사용이 간편 함
- b. 오브젝트의 메소드를 추가시 단축 구문을 이용

4. 배열

- a. 배열을 작성시 리터럴 구문을 사용
- b. 배열을 복사할 때는 `spread` 연산자(`...`)를 이용
 - i. 불필요한 코드 작성을 줄임

5. 문자열

- a. 함수 내에서 문자열을 생성시 `template string`을 이용
 - i. 문자열내 변수 구분이 용이함

```
// bad
function sayHi(name) {
  return "How are you, " + name + "?";
}

// good
function sayHi(name) {
  return `How are you, ${name}?`;
}
```

6. 비교연산자

- a. `'=='` 이나 `'!='` 보다 `'==='`와 `'!=='`를 사용
- b. `if`문과 같은 조건식은 강제형 변환으로 평가되어 아래와 같은 룰을 따름
 - i. `Object` 는 `true` 로 평가된다.
 - ii. `undefined` 는 `false` 로 평가된다.
 - iii. `null` 은 `false` 로 평가된다.
 - iv. `Boolean` 은 `boolean`형의 값 으로 평가된다.
 - v. `Number` 는 `true` 로 평가된다. 하지만 `+0`, `-0`, or `NaN` 의 경우는 `false` 이다.
 - vi. `String` 은 `true` 로 평가된다. 하지만 빈문자 `""` 의 경우는 `false` 이다.

```
1 == '1' // true
1 === '1' // false

0 == [] // true
0 === [] // false
```

7. 형변환

- a. 해당 코드에서 선두에서 형 변환을 수행 (참고 [링크](#))

```
const totalScore = String(this.reviewScore);

const val = parseInt(inputValue);
const val = Number(inputValue);
const val = parseInt(inputValue, 10);

const hasAge = Boolean(age);
const hasAge = !!age;
```

3.4.2. 라이브러리

1. UI를 구현하기 위한 목적으로 템플릿을 사용시
 - a. 템플릿에 포함된 라이브러리를 가급적 수정하지 않고 사용
 - b. 버전 업데이트가 필요할 경우 라이브러리간 충돌이나 스타일링의 문제가 없을 경우 업데이트하여 사용가능
2. 그 외의 경우
 - a. 배포되어 있는 버전중 최신의 안정화 버전을 사용
 - i. 베타, 개발 버전은 사용하지 않음
 - b. 라이브러리는 다운로드 받아 내부에서 로드하도록 사용
 - i. CDN의 관리 할수 없는 요소에 의한 로드 오류를 방지
 - c. 개발 중 라이브러리의 버전이나 타입(uncompressed / minified) 변경 금지
 - i. 변경 후 예측되지 않은 오류 방지
 - ii. 불가피하게 변경 해야 할 경우 내부 검토 필요
3. Modern Admin Template 주요 라이브러리 버전
 - a. [jQuery](#) : v3.3.1
 - b. [jQuery blockUI plugin](#) : 2.70.0-2014.1123
 - c. [Bootstrap](#) : v4.3.1
 - d. [chart.js](#) : v.2.7.1
 - e. [jQuery dataTables](#) : 1.10.16
 - f. 추천 : [File uploader](#) (고려 필요)

3.4.3. jQuery

1. jQuery를 이용해 객체를 가져오는 경우 변수명 앞에 식별자 \$를 표시

```
//bad
const userCheck = $('#userInput');

//good
const $userCheck = $('#userInput');
```

2. DOM 탐색(Selector 사용)

- a. id와 class의 개념에 주의하여 탐색
 - b. 탐색 범위를 좁히기 위한 문맥 정보를 추가하여 탐색
 - c. DOM 계층 관계를 파악 후 탐색
 - d. 전역 selector는 지양
3. 이벤트 처리
- a. HTML 엘리먼트에 직접 이벤트를 추가하지 않음

```
// Good
$('#myLink').on('click', myEventHandler);

<!-- Bad -->
<a id="myLink" href="#" onclick="myEventHandler();" >my link</a>
```

- b. 이벤트 핸들러에 익명함수 사용 금지
- c. 사용자 정의 이벤트는 unbind가 용이하도록 namespace를 사용

```
// Good
$('#myLink').on('click.mySpecialClick', myEventHandler);

// Later on - it's easier to unbind just your click event
$('#myLink').unbind('click.mySpecialClick');
```

4. 메소드 체이닝
- a. jquery 메소드를 연속으로 사용하는 경우 메서드 체이닝을 사용
 - b. 메소드 체이닝 사용시 가독성을 고려하여 줄바꿈 사용

```
$('#myLink')
  .addClass('bold')
  .on('click', myClickHandler)
  .on('mouseover', myMouseOverHandler)
  .show();
```

- c. 같은 메소드를 2번 이상 사용할 경우 객체 리터럴을 사용

```
// Good
$myLink.attr({
  href: '#',
  title: 'my link',
  rel: 'external'
});

// Bad
$myLink.attr('href', '#').attr('title', 'my link').attr('rel', 'external');
```

5. load() 사용

- a. jQuery .load()메소드를 사용시 메소드에 정의된 콜백 함수를 이용

```
$("#result").load("result-table.html", function() {  
    alert("Load was performed.");  
});
```

6. \$(this) 사용 자제 : 의미있는 이름으로 변수 생성

<code>\$(this).prop('tagName');</code>	VS	<code>var addedInput = \$(this); addedInput.prop('tagName');</code>
--	----	---

7. Servlet model의 데이터를 js 혹은 html input에 넘겨줄 때 주의 : 값에 홑따옴표, 쌍따옴표가 있는 경우 오동작 주의

```
var title = "${title}"; // 소스보기하면 아래와 같음  
var title = "as"df"; // title중간에 따옴표가 들어가 있다면 오류 발생  
var title = "<c:out value='${artistNameKo}'/>"; // c:out을 써야 함. 아래 라인 참조  
var title = "as&#034;df";  
  
<input value="${title}" /> // 소스보기하면 아래와 같음  
<input value="as"df" /> // title중간에 따옴표가 들어가 있다면 오류 발생  
<input value="<c:out value='${artistNameKo}'/>" /> // c:out을 써야 함. 아래 라인 참조  
<input value="as&#034;df" />
```

8. css에 정의된 class명으로 참조 지양 👉 디자인 요소 수정시 로직도 수정해야 함

9. 스트링 리터럴 선언시 홑따옴표 사용 👉 가독성 향상

```
const name = 'kim';
```

10. Ajax로 JSON과 첨부파일 같이 저장하기

TODO FormData를 사용해야 함. 샘플 필요

3.4.4. 공통ajax모듈

1. ajax 통신시 비동기 처리로 연결
2. 연결 상태별 메소드를 활용 (아래는 예로 정의)
 - a. beforeSend : Spinner(로딩)을 동작
 - b. success : API에서 정의된 코드별 동작 처리
 - c. complete : Spinner를 닫음
 - d. error : [HTTP상태 코드](#)에 따른 error 처리
 - e. success시 callback 함수로 처리 지향

```
...  
error : function (jqXHR, textStatus, errorThrown) {  
    ajaxErr(jqXHR, textStatus, errorThrown);  
}
```

```
// REST api 호출 후 오류 발생시 공통 처리  
function ajaxErr(jqXHR, textStatus, errorThrown) {
```



```
} ...
```

3.4.5. RESTful API에 post로 JSON객체와 파일첨부를 동시에 전송할 때 샘플

1. front-end : 이걸로..

```
var trackFormData = new FormData();
const trackJson = {
  trackId: trackId,
  msbCode: trackMsbCode,
  isrcCode: isrcCode,
  trackNameKo: trackNameKo,
  ...
};
// JSON모델 추가
trackFormData.append(
  'iTrackInq', // 컨트롤러에 전송할 모델명
  new Blob(
    [JSON.stringify(trackJson)]
    ,{type: "application/json"}
  )
);
// 첨부파일 추가
trackFormData.append("trackFile", document.forms["trackForm_" + trackId].trackFile.files[0]);

$.ajax({
  type: "POST",
  url: "/realse/album/updateTrackInfo",
  processData : false, // 추가
  contentType : false, // 추가
  enctype : 'multipart/form-data', // 추가
  data : trackFormData,
  cache: false,
  beforeSend: function(){
  },
  success: function(data) {
    successAlert();
  },
  complete: function(data) {
  },
  error: function (jqXHR, textStatus, errorThrown) {
    ajaxErr(jqXHR, textStatus, errorThrown);
  }
});
```

a. FormData 사용

b. JSON모델을 문자열로 변환 후 Blob에 담아서 FormData에 추가

2. back-end

```
@ResponseBody
@PostMapping(value = "/realse/album/updateTrackInfo", consumes = {"multipart/form-data"})
public ResultJsonDto updateTrackInfo(HttpServletRequest request, @CurrOzzAdminUser InfoAdminDto
adminUser, @RequestParam TrackInqRsItDto iTrackInq, @RequestPart(value = "trackFile", required =
false) MultipartFile file) {
```

```
} ...
```

- a. 인수를 *@RequestPart*로 받음
- b. consumes 추가

3.5. Database

Database 설정을 위한 규칙 및 네이밍 룰등을 기술한다.

3.5.1. Database 설정

1. Database character sets / collations

Title	Value
Character Sets	utf8mb4
Collations	utf8mb4_0900_ai_ci

Refer to [link](#)

2. 테이블 속성

- character sets / collations: Database 속성과 동일하게 사용한다.
- Engine : innnoDB (transaction지원, 백업 용이성)

3. 가능하면 비즈니스 로직을 사용하지 않는다. (사유: 자동화 단위 테스트가 불가능)

4. 네이밍 표준

네이밍은 Snake Case(대문자, “_” 만 사용)를 기본으로 한다.

a. 테이블명

- 이름은 복수가 아닌 단수로 쓴다.
- 테이블명은 아래 규칙을 따른다.
{그룹명}_{업무명}
- 그룹명은 아래에서 선택 가능하다.

그룹명	설명
INFO	기본적인 정보를 관리하는 테이블 그룹명
STAT	통계관련 정보를 관리하는 테이블 그룹명
CALA	정산관련 정보를 관리하는 테이블 그룹명
RELE	배포관련 정보를 관리하는 테이블 그룹명
BORD	게시판관련 정보를 관리하는 테이블 그룹명
HIST	이력(History)관련 정보를 관리하는 테이블 그룹명
QUEU	큐(Queue)관련 정보를 관리하는 테이블 그룹명

기타	추가가 필요할 경우 DBA에게 요청한다.
----	------------------------

- 업무명은 테이블의 속성을 알 수 있는 단어를 사용하고, 약어보다는 최대한 전체단어를 사용한다.
- Comments는 필수 기재한다.

b. 컬럼명

- 이름은 복수가 아닌 단수로 쓴다.
- 속성을 명확히 알 수 있도록 단어를 사용한다.
- Comments는 필수 기재한다.

c. 인덱스명, FK명

- 인덱스명: `IDX_{테이블명}`, `IDX_{테이블명}_{순번}`
- 외래키명: `FK_{본인 테이블명}_{부모 테이블명}`
- ~~FK는 지양한다.~~

d. function명

- 규칙: `FC_{업무명}`
- 단, function 사용은 지양한다.

3.5.2. DB관리 추천 툴

1. ERD편집기 : [Mysql Workbench](#)

3.6. 인증(Authentication)과 인가(Authorization)

Spring Security 사용 *TODO*상세내용추가

3.7. 반영

4. 산출물 관리

프로젝트가 완료 되었을때 취합되어야할 산출물들을 나열한다.
산출물에는 문서, source 등 각종 형식이 포함된다.

4.1. 기획

산출물 리스트

- Workflow Define (Flow Chart)
- Requirement Statement
- Information Architecture
- Design Document (Wireframe / Process / Functional Spec.)
- Etc. (Code Matrix, Message Template...)

4.2. 설계

산출물 리스트

4.3. 개발

산출물 리스트

- 프로그램 리스트
- 개발 소스(zip파일)
- Java Doc : 📌 [샘플](#)
TODO 설정하기 추가

5. 유지/보수

5.1. 태스크월드

등록 → 검토 → 진행 → 완료

1. 등록 : 누구나 등록자가 될 수 있음. 등록시 담당자를 배정해야 함. 추후 처리결과를 검토하고 완료처리 진행.
2. 검토
 - a. 담당자는 해당 업무 예상 처리 소요 시간 산정 후 업무 업데이트
 - b. 담당자와 등록자가 포인트(중요도/심각성)와 일정 협의 후 업무 업데이트
3. 진행 : 가능하면 git브랜치를 만들어서 작업(생성한 브랜치는 공유가 필요할 때만 push함)
 - a. 브랜치명
 - i. 버그 수정 : **bugfix_#태스크월드업무번호**
 - ii. 등록 → 검토 → 진행 → 완료
 - iii. 등록 : 누구나 등록자가 될 수 있음. 등록시 담당자를 배정해야 함. 추후 처리결과를 검토하고 완료처리 진행.
 - iv. 검토
 - v. 담당자는 해당 업무 예상 처리 소요 시간 산정 후 업무 업데이트
 - vi. 담당자와 등록자가 포인트(중요도/심각성)와 일정 협의 후 업무 업데이트
 - vii. 진행 : 가능하면 git브랜치를 만들어서 개발
 - viii. 기능 추가 : **feat_업무번호**
 - b. 진행 중 질의는 태스크월드를 이용.
 - c. 수정완료 시 소스를 **push**후 운영에 반영하고 태스크월드에 코멘트를 추가하면 관련자들에게 알림이 발송됨
4. 완료 : 등록자는 처리결과를 검토한 후 완료처리를 하거나 미진한 부분에 대한 코멘트를 추가하면 관련자들에게 알림이 발송됨

※ 해당 업무 반영 후 버그 발생 시 해당 브랜치를 다시 사용해서 디버깅(master에서 바로 수정하지 마세요)

5.2. 개발 서버

- git에 develop 개발서버 배포용 브랜치가 있음. 여기에 테스트 할 기능들을 commit 후 반영 & 운영자 테스트
- ~~사외에서 테스트 사이트 접근 불가~~ 🙌 사내가 아니라 AWS에 넣을 예정
- 

6. 기타

1. 문법 강조 하기 <https://tohtml.com/html/>
2. spring boot 용 banner.txt 만들기
 - a. 참고 : [Spring Boot Banner.txt \(프로젝트 타이틀 콘솔 출력\)](#)
 - b. [배너 샘플 보기](#)